

FACTORY PATTERN

(Creational Design Pattern)

1 Mục tiêu tuần học

Sau bài này, sinh viên có thể:

- Hiểu vấn đề thực tế mà Factory Pattern giải quyết
- Phân biệt new object trực tiếp vs tạo object qua Factory
- Áp dụng Factory để:
 - Ẩn logic khởi tạo phức tạp
 - Giảm coupling trong business code
 - Kết hợp Factory + Strategy + DI
- Nhận diện vai trò Factory trong Clean Architecture

2 Vấn đề thực tế (Motivation)

- Bối cảnh Order Management. Hệ thống hỗ trợ nhiều hình thức thanh toán:
 - PayPal
 - VNPay
 - Stripe
 - Credit Card
 - Bank Transfer
 - Crypto (tương lai)
- ...

👉 OrderService cần một PaymentStrategy phù hợp theo loại đơn hàng / cấu hình / user chọn

3 Thiết kế ngân thờ

```
public IPaymentStrategy CreatePayment(string type)
{
    if (type == "Paypal")
        return new PaypalPayment();
    if (type == "VNPay")
        return new VNPayPayment();
    if (type == "Stripe")
        return new StripePayment();
    throw new Exception("Unsupported payment");
}
```

🔗 Vấn đề:

Vi phạm OCP (thêm payment → sửa code)

Logic tạo object bị rải rác

OrderService biết quá nhiều về chi tiết khởi tạo

➡ Cần Factory Pattern

4 Định nghĩa Factory Pattern

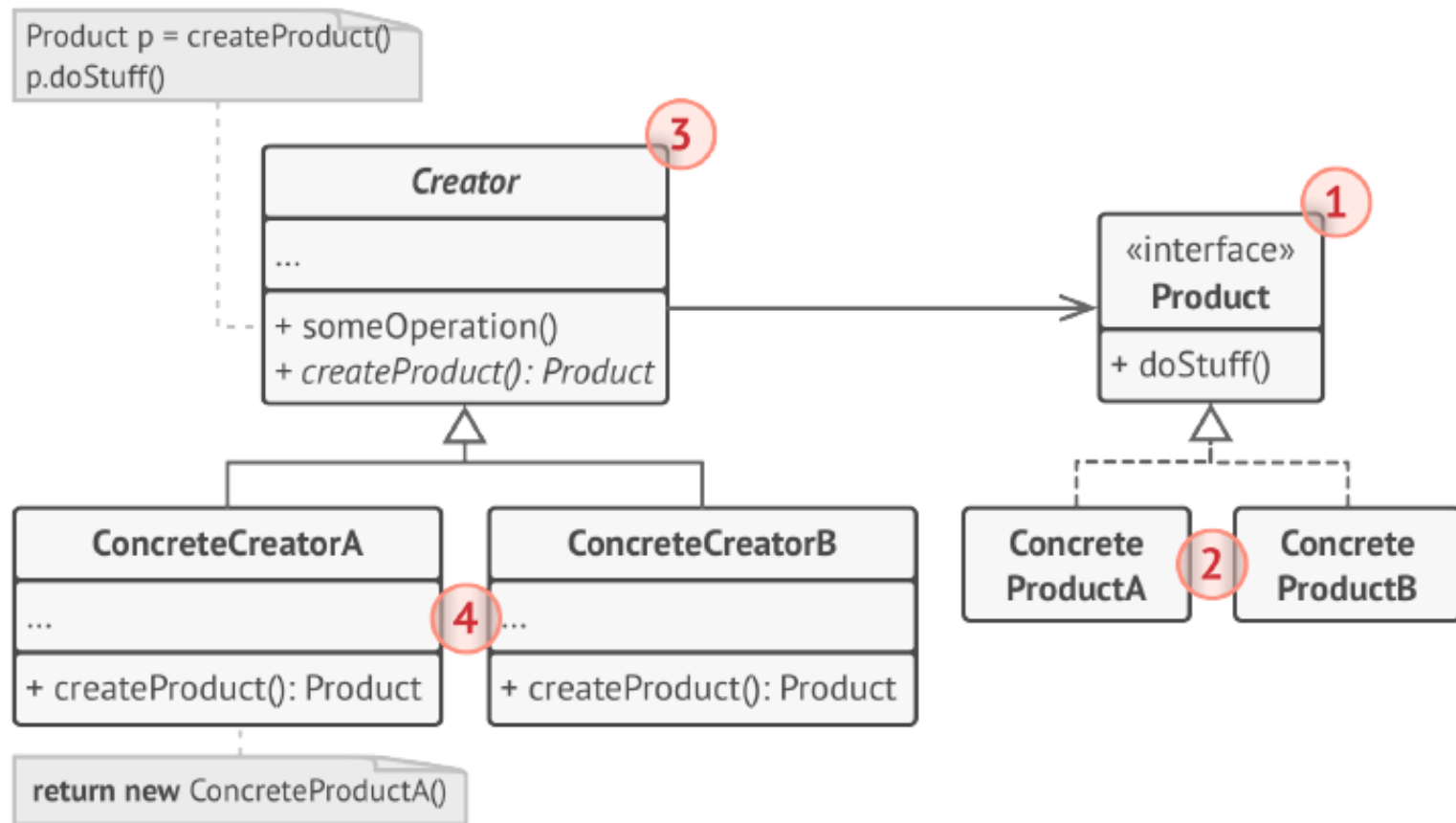
Factory Pattern

- Cung cấp một interface để tạo object nhưng ủy quyền việc quyết định class cụ thể cho một đối tượng khác.

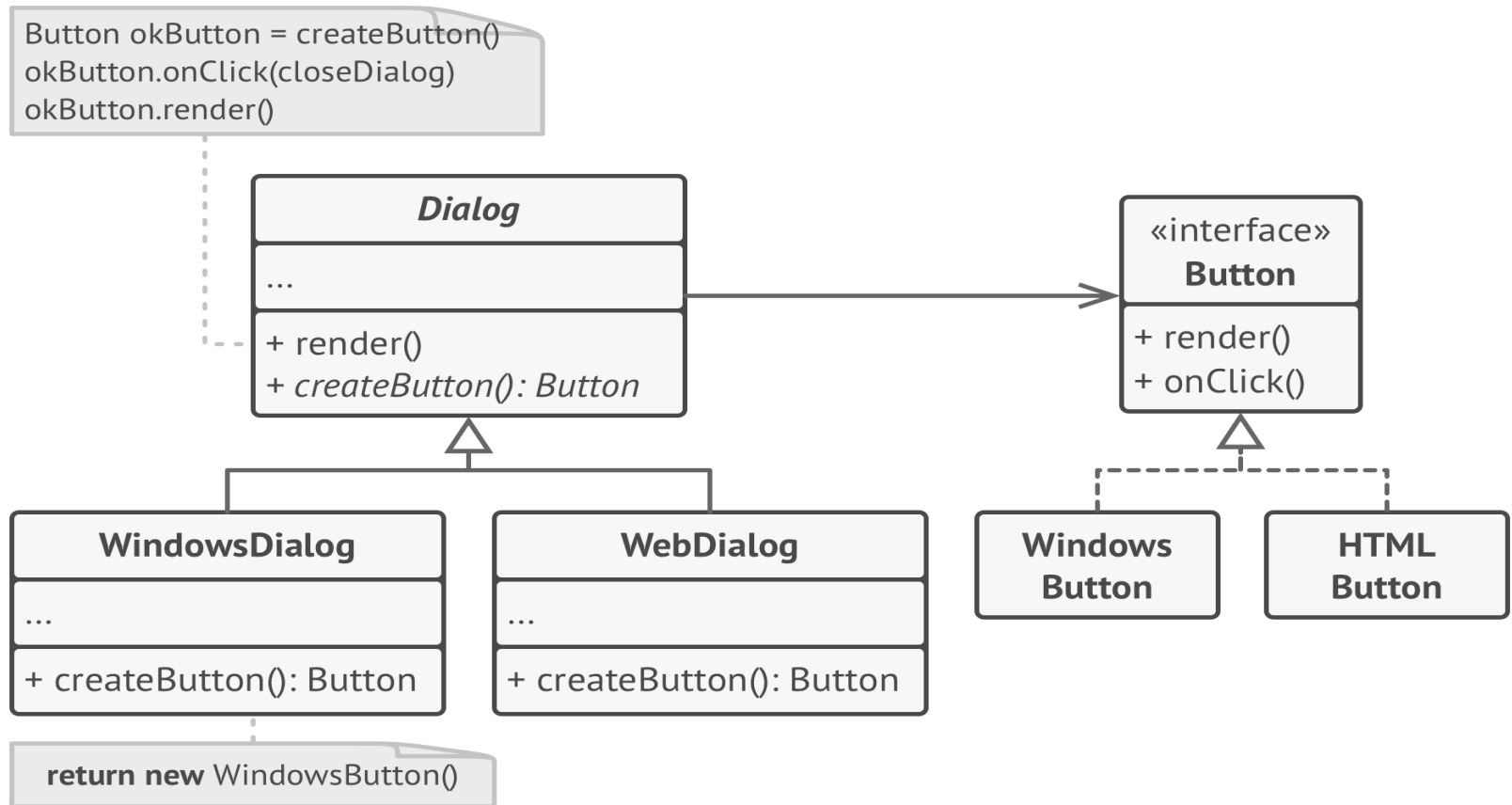
 Nói đơn giản:

“ Business code không new object
Factory chịu trách nhiệm tạo ”

5 Cấu trúc Factory Pattern



5 Ví dụ Factory Pattern



6 Áp dụng vào Order Management

Bài toán

Hệ thống xử lý thanh toán cho đơn hàng.

Yêu cầu:

- Hỗ trợ nhiều loại thanh toán
- Mỗi loại có cách xử lý riêng
- Dễ mở rộng trong tương lai

Bước 1: Product Interface

```
public interface IPayment
{
    void ProcessPayment(decimal amount);
}
```

Bước 2: Concrete Products

```
public class PaypalPayment : IPayment
{
    public void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing {amount} via PayPal");
    }
}
```

```
public class VNPayPayment : IPayment
{
    public void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing {amount} via VNPay");
    }
}
```

Bước 3: Creator (Quan trọng nhất)

- *Checkout() là business logic*
- *CreatePayment() là factory method*
- *Creator không biết concrete class nào được tạo*

```
public abstract class PaymentCreator
{
    // Factory Method
    public abstract IPayment CreatePayment();

    // Business Logic dùng Factory Method
    public void Checkout(Order order)
    {
        var payment = CreatePayment();
        payment.ProcessPayment(order.FinalAmount);
    }
}
```

Bước 4: Concrete Creators

```
public class PaypalPaymentCreator : PaymentCreator
{
    public override IPayment CreatePayment()
    {
        return new PaypalPayment();
    }
}

public class VNPayPaymentCreator : PaymentCreator
{
    public override IPayment CreatePayment()
    {
        return new VNPayPayment();
    }
}
```

Bước 5: Client sử dụng

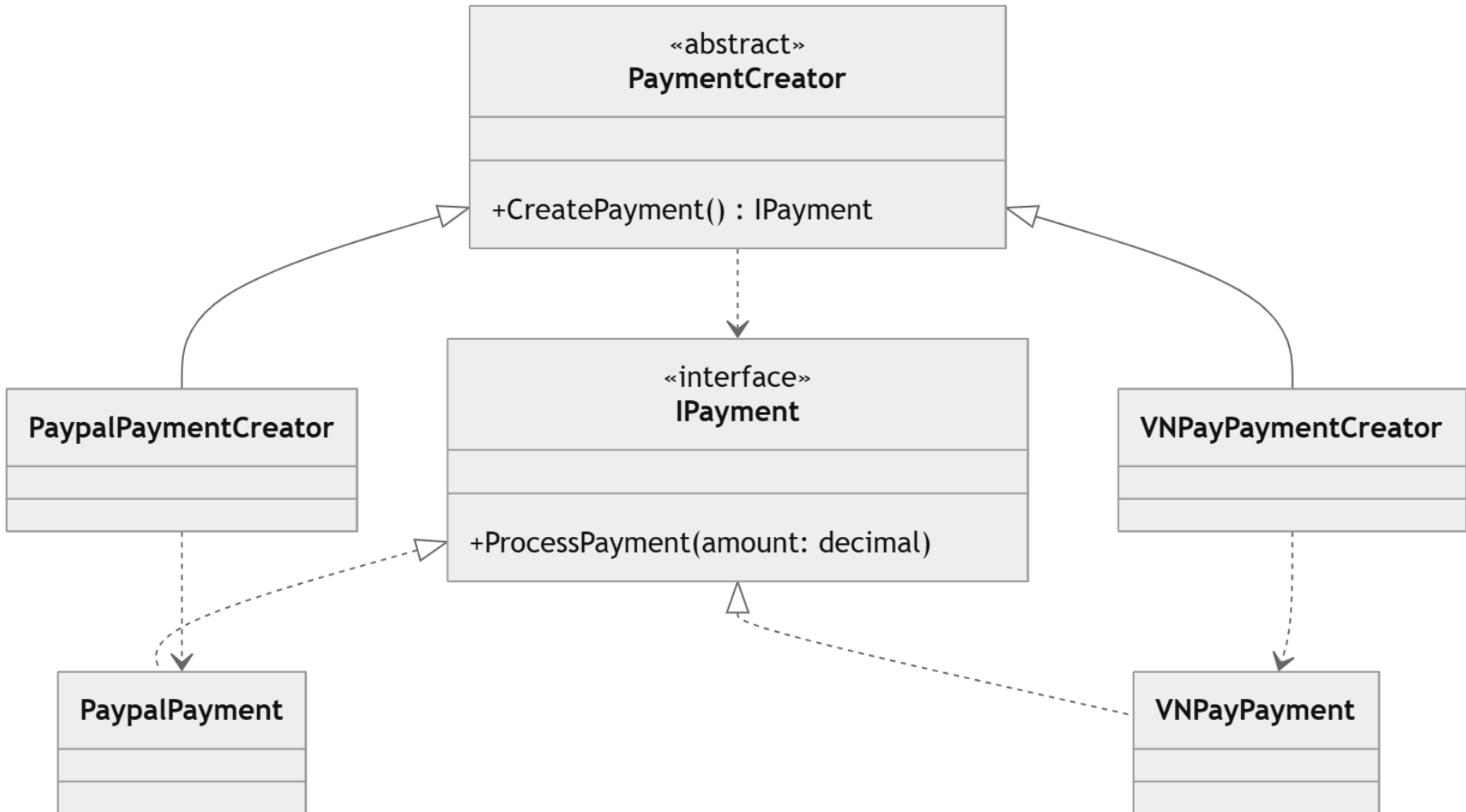
```
var order = new Order { FinalAmount = 500 };
```

```
PaymentCreator creator = new PaypalPaymentCreator();  
creator.Checkout(order);
```

Hoặc:

```
PaymentCreator creator = new VNPayPaymentCreator();  
creator.Checkout(order);
```

Class Diagram trong hệ thống Order Management



7 Thực tế: Kết hợp Factory + DI

Sử dụng DI container:

```
AddTransient<PaymentCreator, PaypalPaymentCreator>()
```

```
AddTransient<PaymentCreator, VNPayPaymentCreator>()
```

Đăng ký PaypalPaymentCreator vào DI container với vòng đời Transient.

Mỗi lần hệ thống cần, container sẽ tạo một instance mới và inject vào nơi cần dùng

7 Thực tế: Kết hợp Factory + DI

```
public abstract class PaymentCreator
{
    public abstract bool CanHandle(string paymentType);
    public abstract IPayment CreatePayment();
}
```

```
public class PaypalPaymentCreator : PaymentCreator
{
    public override bool CanHandle(string paymentType)
        => paymentType == "Paypal";

    public override IPayment CreatePayment()
        => new PaypalPayment();
}
```


7 Thực tế: Kết hợp Factory + DI

```
public class OrderService
{
    private readonly IEnumerable<PaymentCreator> _creators;

    public OrderService(IEnumerable<PaymentCreator> creators)
    {
        _creators = creators;
    }

    public void Checkout(string paymentType)
    {
        var creator = _creators
            .First(c => c.CanHandle(paymentType));

        var payment = creator.CreatePayment();
        payment.ProcessPayment(1000);
    }
}
```

8 Khi nào dùng Factory?

✓ Khi:

- Framework
- Plugin system
- Provider model
- Khi business logic phụ thuộc product
- Khi muốn mở rộng mà không sửa code cũ

✗ Không cần dùng khi:

- Chỉ có 1 implementation
- new object quá đơn giản và không thay đổi

9 Các loại Factory

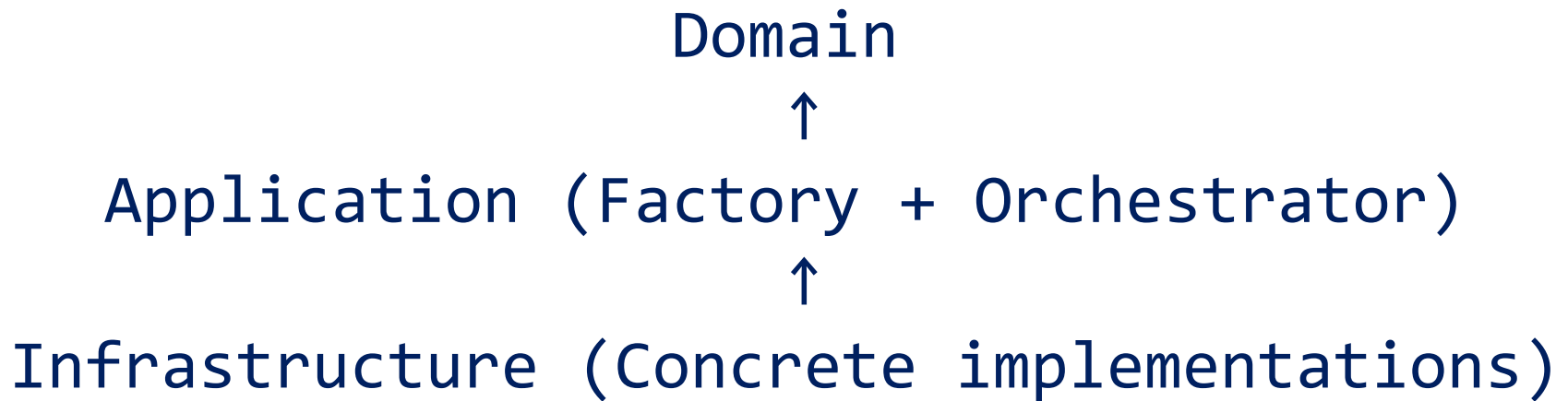
Pattern	Mục đích
Simple Factory	Tập trung logic tạo
Factory Method	Cho subclass quyết định tạo
Abstract Factory	Tạo họ object liên quan

10 Factory vs Strategy (so sánh)

Strategy	Factory
Thay đổi hành vi	Tạo object
Runtime behavior	Creation logic
Giải quyết OCP	Giải quyết coupling khi new
Dùng trong business	Dùng ở biên khởi tạo

1 1 Vai trò Factory trong Clean Architecture

📌 Factory thường nằm ở:
Application layer
Hoặc Infrastructure (tùy mức độ)



1 2 Bài tập trên lớp

 Bài 1: Mở rộng hệ thống có chính sách giảm giá theo loại tài khoản VIP, Student.

Viết DiscountFactory tạo:

- VipDiscount
- StudentDiscount

 Bài 2

Vẽ UML Before / After khi áp dụng Factory

1 3 Câu hỏi tư duy

- Factory có vi phạm OCP không?
- Khi nào nên bỏ Factory, dùng DI trực tiếp?
- Factory có phải luôn tốt không?
- Vì sao Factory giúp code “sống lâu”?

1 4 Kết luận

“ Strategy quyết định làm gì ”

“ Factory quyết định tạo cái gì ”

“ Business code không nên biết tạo như thế nào ”